2025 EDITION

Claude Code 完全指南

从零到一实操手册 · 涵盖安装配置、三种模式、核心命令、MCP/Skills/Hooks/Agent，以及Token节省技巧

实操手册

2025年4月

25

目录

01 环境准备——开始前必读

02 安装 Claude Code

03 配置 CCSwitch（国内用户必做）

04 启动与界面

05 三种运行模式详解

06 init 初始化

07 创建 Agent

08 MCP vs Skills——搞清楚区别

09 Hooks 钩子

10 隐藏命令精讲

11 Plugins

12 避免长对话 Bug

13 节省 Token

14 设计自己的 Skill

01 环境准备——开始前必读

> **Claude Code 不只是写代码！**
>
> 它是一个万能 AI Agent，可以做数据分析、管理文件、爬取资讯、修图生图、剪辑视频，以及执行你自己设计的自动化工作流。

1.1 确认操作系统与终端

macOS / Linux：直接使用系统自带终端（Terminal / iTerm2）。

Windows：强烈推荐使用 PowerShell，而不是 CMD。Claude Code 底层是 Unix 风格命令，PowerShell 兼容性更好。

可在开始菜单搜索「PowerShell」打开，或右键桌面选「在终端中打开」。

1.2 确保 Node.js 已安装

Claude Code 依赖 Node.js 环境。在终端中执行以下命令，看是否有版本号输出：

终端

```
node --version
# 应输出类似 v18.x.x 或更高版本
```

如果提示「command not found」，请先访问 nodejs.org 下载 LTS 版本安装。

02 安装 Claude Code

2.1 访问官网获取安装命令

打开 claude.ai/code（或搜索「Claude Code docs」），向下滚动找到安装命令区域，根据系统选择对应命令。

2.2 macOS / Linux 一键安装

MACOS / LINUX 终端

```
npm install -g @anthropic-ai/claude-code
```

粘贴命令后回车，等待自动安装完成即可。

2.3 Windows 安装

在 PowerShell 中执行：

POWERSHELL（管理员）

```
npm install -g @anthropic-ai/claude-code
```

权限问题

若提示权限错误，请右键 PowerShell → 「以管理员身份运行」，再执行命令。

2.4 验证安装成功

终端

```
claude --version
# 输出版本号说明安装成功
```

03 配置 CCSwitch（国内用户必做）

> **CCSwitch 的作用**
>
> 国内访问 Anthropic 官方服务不稳定，CCSwitch 允许你配置多个国产模型（MiniMax、DeepSeek、Kimi、百变、通义等）并一键切换，作为 Claude Code 的后端。

3.1 macOS 安装 CCSwitch

MACOS 终端

```
# 在 CCSwitch 官网「快速开始」区域复制 macOS 安装命令
# 通常格式如下（以实际官网命令为准）：
brew install ccswitch
# 或
curl -fsSL https://ccswitch.dev/install.sh | bash
```

3.2 Windows 安装 CCSwitch

Windows 用户需手动下载安装包：

1. 访问 CCSwitch GitHub 仓库，点击 **Releases**
2. 展开 **Assets** 列表
3. 下载 **CCSwitch-v3.x.x-windows.msi**（注意必须是 .msi 格式）
4. 双击运行安装程序，按提示完成安装

3.3 添加模型（以 MiniMax 为例）

打开 CCSwitch 界面后：

1. 点击右上角 「+」 按钮
2. 在模型列表中选择 **MiniMax**（其他模型同理）

3. 系统会自动填好 **Base URL**，无需手动填写
4. 只需填入你的 **API Key**
5. 点击「添加」完成配置

> **如何获取 MiniMax API Key**
>
> 访问 MiniMax 官网 → 账户管理 → Token 管理页面，复制你的 API Key 粘贴进来即可。建议先充值 49元套餐，性价比较高。

3.4 验证配置

启动 Claude Code 后，界面左上角会显示当前使用的模型名称（如 minimax-2.7）和 API 状态，说明配置成功。

04 启动与界面

4.1 进入工作目录

先用 cd 命令进入你想操作的文件夹：

终端

```
# 示例：进入桌面
cd ~/Desktop

# 或进入某个项目文件夹
cd ~/projects/my-project
```

4.2 启动 Claude Code（普通模式）

终端

```
claude
```

首次启动会询问是否信任当前文件夹作为 workspace，选择 Trust Folder 即可。

4.3 启动 Claude Code（危险模式 / 全自动模式）

终端

```
claude --dangerously-skip-permissions
```

危险模式说明

此模式下 Claude Code 执行任何操作（创建/删除/修改文件、运行命令）不再询问你的确认，直接自动执行。适合任务明确、信任度高的场景，可节省大量点击操作。

⚠️ **建议在非重要文件夹下使用，避免误删重要数据。**

4.4 推荐输入法：语音输入提效

视频作者推荐「闪电说」语音输入法——基于大模型整理语音内容，识别准确率高，可大幅减少打字时间。抖音的「豆包输入法」也是类似工具，均免费可用。

05 三种运行模式详解

模式	说明	适用场景
DEFAULT MODE	每次执行操作都会暂停等待确认	谨慎操作场景
PLAN MODE	AI 先读取信息，生成执行计划供审核	复杂任务、批量操作
BYPASS	跳过所有权限确认，全程无需干预	任务明确、批量操作

5.1 默认模式

每次执行操作（读/写/删文件、运行命令）都会暂停等待你的确认。安全但需频繁点击。适合谨慎操作场景。

5.2 规划模式

AI 先只读取信息，生成一份执行计划给你审核。确认后才开始实际写操作。适合复杂任务、大量文件操作或代码 Review。

5.3 全自动模式

跳过所有权限确认，全程无需人工干预。启动参数：`--dangerously-skip-permissions`。适合任务明确、批量操作场景。

5.4 如何切换到 Plan Mode

在 Claude Code 界面内，使用 **Shift** + **Tab** 在不同模式之间循环切换。

5.5 Plan Mode 三个选项的含义

1. **Yes (bypass)** — 确认计划，后续无需再次确认，直接执行
2. **Manual approve/edit** — 确认计划，但每步操作仍需手动批准
3. **修改计划** — 用自然语言告诉 AI 你想调整什么，AI 会更新计划后再次让你审核

06 init 初始化

6.1 执行 init 命令

CLAUDE CODE 内

```
/init
```

Claude Code 会扫描整个项目代码，理解项目结构，然后自动生成一份 CLAUDE.md 文件。

6.2 CLAUDE.md 是什么

CLAUDE.md 是 Agent 的「最高指令文件」，每次对话开始时会自动加载。你可以在里面写：

- 开发规范（代码风格、命名规则等）
- 禁止执行的危险操作（如禁止 `rm -rf`）
- 项目背景和架构说明
- 你的个人偏好和工作习惯

6.3 CLAUDE.md 最佳实践：渐进式披露

> **切忌把 CLAUDE.md 写得又臭又长！**
>
> 过长的文件会：
>
> 1. 每次对话消耗大量 token
> 2. 让模型「迷失方向」，变得不聪明

推荐做法：把详细内容拆分到独立 MD 文件，在 CLAUDE.md 中只写引用路径：

CLAUDE.MD 示例

```
# 项目概述（简短）
这是一个电商后台系统，使用 React + Node.js。

# 开发规范
详见 docs/dev-guide.md，需要时请查阅该文件。

# 禁止操作
- 禁止执行 rm -rf 命令
- 不得修改 .env 文件
```

6.4 什么时候运行 init

任何时候进入一个新项目都应该运行 `/init`，尤其是：

- 从 GitHub 下载了别人的开源项目
- 接手一个新团队的项目
- 自己的项目经过大规模重构后

07 创建 Agent

> **Agent 本质**
>
> Agent 是一个拥有特定职责、专属 Prompt 和固定工具权限的「角色」。它能处理同类任务、节省上下文，还可以和其他 Agent 自动协作。

7.1 进入 Agent 创建界面

CLAUDE CODE 内

```
/agent
```

选择 **Create new agent**，然后：

- **Location**：选 Project（项目级别权限）
- **创建方式**：选「Generate with Claude」（AI 辅助生成，推荐）

7.2 描述 Agent 职责

用自然语言描述这个 Agent 要做什么。越清晰越好，例如：

AGENT 描述示例

> 你是一个技术合伙人 Agent。
> 你的职责：
> 1. 评估产品需求的技术可行性
> 2. 设计系统架构方案
> 3. 拆解开发任务，给出时间估算
> 4. 代码 Review，确保工程质量
> 5. 识别技术风险并给出应对方案
> 请使用简洁专业的中文回答。

7.3 配置工具权限、模型、颜色

- **工具权限**：可全选或按需选择（↑ ↓ 导航，Enter 切换）
- **模型**：选 `Inherit` 继承父级模型即可
- **颜色**：选一个你喜欢的颜色便于区分
- **Memory 范围**：选当前项目级别

7.4 调用已创建的 Agent

直接用自然语言呼唤即可：

对话输入

> 帮我调用技术合伙人 Agent，我想讨论一个新项目的技术选型

7.5 什么时候该建 Agent

满足以下条件时就该建专属 Agent：

- 你会**重复执行同一类任务**
- 这类任务需要**很长的 Prompt** 和大量规范约束
- 任务结果不需要你关注中间过程，只看最终输出

> **常见 Agent 团队示例**

> 产品经理 Agent、后端开发 Agent、前端开发 Agent、测试 Agent、代码 Review Agent。甚至可以建一个 Codex Agent，让两个 AI 自动协作，无需你传递上下文。

08 MCP vs Skills——搞清楚区别

概念	作用	类比
MCP	告诉 AI「能不能做」——给 AI 接入外部能力	「长出了腿」
Skills	告诉 AI「怎么做」——按什么步骤、规范去执行	「骑车指南」

> **开卷考试类比**
>
> **MCP** = 你有没有带参考书进考场（有没有工具权限）
> **Skills** = 你怎么高效查书找答案（如何使用这条腿）
>
> 两者结合才能发挥最大效果。

8.1 查看已安装的 MCP 和 Skills

CLAUDE CODE 内

```
# 查看已安装的 MCP
/mcp

# 查看已安装的 Skills
/skills
```

8.2 Skills 越精越好，不是越多越好

> **工具调用选择恐惧症**

> Skills 过多 = 模型有太多工具可选，反而不知道何时该用哪个，导致执行效果变差。

建议：在使用过程中发现哪些工作流会重复，再针对性地创建，并持续精简无用 Skill。

09 Hooks 钩子

> **Hooks 本质**
>
> 在特定事件发生前/后，自动执行一段脚本。类比：监控摄像头（前置 hook）或自动报警器（后置 hook）。

9.1 两大类 Hooks

- **工具调用前（Pre-tool Hook）**：AI 执行操作之前触发，可用于拦截、过滤
- **工具调用后（Post-tool Hook）**：AI 执行操作之后触发，可用于通知、同步

9.2 应用场景示例

Hook 类型	功能
隐私保护 Hook	每次 AI 读文件前，检查路径是否包含私密文件（如 .env、密钥文件），若匹配则拦截操作
自动推送 Hook	每次代码 Review 完成后，自动触发 git push 推送到 GitHub
通知 Hook	长任务完成后，自动发送飞书/钉钉通知

9.3 创建 Hook

直接用自然语言告诉 Claude Code 你的需求，它会引导你完成配置：

对话示例

```
帮我创建一个 Hook：
每次 AI 完成代码修改后，自动执行 git add . && git push
推送到我的 GitHub 仓库
```

10 隐藏命令精讲

命令	功能说明	典型场景
`/simplify`	自动 Code Review。派出 3 个子 Agent 分别检查重复代码、代码质量、性能隐患	每次完成一批代码修改后运行
`/rewind`	时光倒流。回到之前的某个 checkpoint，同时恢复代码和上下文	项目做歪了、走错方向时
`/insights`	生成协作洞察报告。分析使用习惯、时间分布、决策模式等	定期复盘优化工作方式
`/loop`	设置定时循环任务（3天内有效）	每小时抓取 AI 新闻、定时检查任务状态

10.1 /simplify 使用前提

运行 `/simplify` 要求代码已托管在 GitHub。它会先执行 `git diff` 查看与上次提交的差异，再派三个子 Agent 审查变更部分。若还没用 Git，告诉 Claude Code「帮我把代码上传到 GitHub」，它会全程协助。

10.2 /loop + Skill 组合示例

对话示例

```
# 每小时执行一次「AI新闻采集」Skill
/loop interval=1h 执行「AI资讯采集」Skill，把结果保存到 ~/ai-news/today.md
```

11 Plugins

> **Plugin 定义**
>
> Plugin = Hooks + Skills + MCP 的打包封装。方便团队成员一键安装复用同一套工作流配置，无需逐个手动配置。

11.1 安装 Codex Plugin（官方出品）

Codex 官方发布了 claude-plugin-cc，可在 Claude Code 中直接调用 Codex，两个 AI 自动协作：

终端

```
# 安装插件
npm install -g claude-plugin-cc

# 在 Claude Code 内加载
/setup codex
```

安装完成后，Claude Code 会自动读取你桌面上的 Codex 配置并写入，即可在 Claude Code 中直接唤起 Codex 协同工作。

12 避免长对话 Bug

> **问题根源**
>
> 对话越来越长，Claude Code 会自动压缩上下文（autocompact）。多次压缩后，早期的关键决策、架构约束会被遗忘，导致越来越多的莫名 Bug 出现。

12.1 写入 CLAUDE.md 的核心决策

把项目中「一旦忘记就会出问题」的关键决策固化到 CLAUDE.md，每次对话都会自动加载：

- 架构约束（如「前端一律用 React，禁止引入 Vue」）
- 已验证的技术方案
- 已否定的路径（避免 AI 再次走弯路）

12.2 使用 Handoff 文件交接上下文

在旧对话结束前，让 AI 生成一份 handoff.md，然后在新对话中加载它：

对话指令

```
请帮我生成一份 handoff.md，包含：
1. 当前进展（已完成什么）
2. 下一步目标
3. 已验证有效的方案
4. 已排除的方案（及原因）
5. 当前遗留问题
```

新对话开始时：

新对话开头

```
请先阅读 handoff.md，然后继续我们的项目开发。
```

13 节省 Token

13.1 安装 RTK Skill

工具调用（如运行测试、搜索文件）会产生大量输出内容，而 AI 其实只需要知道「成功/失败」。RTK Skill 会自动压缩这些输出，只返回关键信息给模型，显著节省 token。

对话指令

> 帮我安装并配置 `RTK Skill`，用于压缩工具调用的输出内容

13.2 其他 Token 节省技巧

技巧	说明
CLAUDE.md 精简	使用渐进式披露（引用外部文件）
子任务独立化	复杂子任务交给独立 Agent 执行，主对话只接收结果
主动开新对话	长对话到一定程度主动开新对话 + Handoff 文件接续
Plan Mode 规划	先规划确认后再执行，避免走弯路浪费 token

14 设计自己的 Skill

> **什么时候创建自定义 Skill?**
>
> 当你发现某个工作流会高频重复，且需要固定的规范约束，就是创建 Skill 的好时机。

14.1 Skill 的本质

Skill 本质上就是一个结构化的 Prompt 模板，通常是一个 Markdown 文件，描述：

- **触发条件**：什么情况下使用这个 Skill
- **执行步骤**：具体怎么做，分几步
- **输出规范**：结果保存到哪里、格式要求
- **注意事项**：禁止做什么

14.2 视频作者案例：录素材 Skill

- **触发条件**：收到一个视频链接（抖音/小红书/YouTube/B站）
- **执行步骤**：① 抓取视频字幕 → ② 整理成脚本稿件 → ③ 保存到本地指定位置
- **效果**：发一个链接，自动完成视频脚本提取，无需手动操作

14.3 快速创建 Skill 模板

对话指令

```
帮我创建一个 Skill，命名为「daily-news-collector」：
- 触发：我说「抓取今日 AI 新闻」
- 步骤：搜索今日 AI 相关新闻 → 提炼5条重点 → 生成摘要
- 输出：保存到 ~/daily-notes/YYYY-MM-DD-news.md
- 格式：每条新闻含标题、要点、来源链接
```

Claude Code 完全指南 · 基于视频内容整理

工具只是工具，Dream Bigger 才是核心